⚠ **TRAINING EXAMPLE — NOT A REAL IEC 62304 DELIVERABLE**

This project is a **training site, not a real medical device** under any regulatory definition — it is not Software as a Medical Device (SaMD) or Software in a Medical Device (SiMD). This page is not a genuine IEC 62304 deliverable: it illustrates the *kind* of document a real SaMD/SiMD project would produce. See the document register for the full explanation.

Clause: 8 — Software Configuration Management · **Register:** Document Register

What the standard requires (Class C — every sub-clause of Clause 8)

REF	TITLE	APPLIES AT	OUTPUT
8.1.1	Establish means to identify configuration items	A, B, C	Scheme for unique identification of configuration items and their versions
8.1.2	Identify SOUP	A, B, C	For each SOUP item: title, manufacturer, unique SOUP designator
8.1.3	Identify system configuration documentation	A, B, C	The set of configuration items and versions comprising the software system configuration, documented
8.2.1	Approve change requests	A, B, C	An approved change request for every change to a controlled configuration item
8.2.2	Implement changes	A, B, C	—
8.2.3	Verify changes	A, B, C	—

REF	TITLE	APPLIES AT	OUTPUT
8.2.4	Provide means for traceability of change	A, B, C	Records of relationships between change request, relevant problem report, and change approval
8.3	Configuration status accounting	A, B, C	Retrievable records of the history of controlled configuration items, including the system configuration

8.1.1 — What is controlled, and how it's identified

Git is this project's configuration management system, in full: every source file, data file, test, and this document set itself is version controlled. The identification scheme is git's own — a commit SHA uniquely identifies the state of every file at that point — supplemented by ordinary file paths, which is sufficient at this project's scale (a handful of pages, no parallel branches of released product).

CONFIGURATION ITEM CATEGORY	EXAMPLES	IDENTIFIED BY
Source	`*.html` , `*.js` , `style.css`	Git path + commit SHA
Content	`data/*.json`	Git path + commit SHA
Tests	`tests/test_site.py` , `tests/capture_screenshots.py`	Git path + commit SHA
Test-only tools	Playwright, axe-core	See SOUP table below
Generated records	`tests/anomaly_log.csv` , `docs/assets/screenshots/*.png`	Git path + commit SHA — committed rather than gitignored, so their own history is retrievable (8.3)
Documentation	`DESIGN.md` , `README.md` , `docs/*.md`	Git path + commit SHA

8.1.2 — SOUP identification

SOUP ITEM	MANUFACTURER	UNIQUE DESIGNATOR
Playwright	Microsoft	`playwright>=1.40` (`tests/requirements.txt`)
axe-core	Deque Systems	`4.10.2` , pinned in the CDN URL constant `AXE_CDN` in `tests/test_site.py`

(See 04 — Architecture for why Formspree is deliberately **not** listed here — it is an external service, not a SOUP component under this project's configuration control.)

8.1.3 — System configuration documentation

The complete set of configuration items comprising a given version of the site is, precisely, the state of the git working tree at a given commit — there is no separate manifest to keep in sync with it, which removes an entire category of configuration drift by construction.

8.2.1–8.2.4 — Change control

For a solo-maintained project, "approval" is a single reviewer decision rather than a change control board — but it is not undocumented:

1. **Change request:** a problem identified via 13 — Problem Resolution (an `ANOM-####` entry) or a feature request from 10 — Maintenance Plan.
2. **Implementation:** made against the actual files, following 02 — Software Development Plan.
3. **Verification:** `tests/test_site.py` must pass before a change is considered complete — see 08.
4. **Traceability (8.2.4):** the git commit message is the link between change and rationale — this project's commit convention states *why* a change was made, not only what changed, matching the "comment the why" discipline used throughout the code itself.

8.3 — Configuration status accounting

`git log` is the retrievable history 8.3 asks for — every configuration item's complete change history, by whom (in the sense of which commit), and when, is queryable directly. As of this document, the repository holds 34 commits and **no version tags** — the tagging gap is the same one recorded in 09 — Software Release, since a tag is what would let 8.3's "history of the system configuration" be queried at named release boundaries rather than only at arbitrary commits.

Gaps

- **No formal baseline/tag scheme** — see 09. Everything else in this plan is in place; this is the one piece that depends on a decision (when to cut a release) rather than a mechanism this project already has.
- **No documented recovery/rollback procedure** — git makes rollback technically trivial (`git revert`), but there is no written procedure describing when a rollback is the right response to a defect versus a forward fix, which a stricter CMP would specify.